

## GenAI handson-2 Observations

### Langchain

---

*Name: Anushree Sanjeev Sawant*

*SRN: PES2UG23CS911*

*Section: B*

---

#### 1.Langchain foundation

Instead of calling a specific API (like Google Gemini or OpenAI) directly, LangChain acts as a Universal Adapter.

Problem: If you use a specific API directly, switching models later requires rewriting the entire codebase because every provider has different response formats and parameters.

Solution: LangChain provides a standard interface. You write your logic once, and switching from Gemini to another model often requires changing only one line of code.

```
- **Without LangChain:** You have to rewrite all your API calls  
(different endpoint, different parameters, different response  
format) .  
  
- **With LangChain:** You change **one line of code**.
```

A token can be a whole word, a part of a word (like "ing"), or punctuation.

Temperature- randomness in the text

Temperature 0.0 (The Accountant): The model always chooses the most likely next word. This results in deterministic, consistent, and factual responses.

Temperature 1.0 (The Poet): The model takes risks, choosing less likely words. This results in creative, varied, and diverse responses.

```

▶ prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")

... --- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or mental image formed in the mind.
Run 2: An idea is a thought, concept, or mental image formed in the mind.

▶ print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")

... --- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, or plan formed in the mind, often serving as a basis for understanding, action, or creation.
Run 2: An idea is a thought, concept, or plan originating in the mind.

```

LLMs understand specific roles that dictate how they should behave:

- System Message: High-level instructions that set the "persona" or constraints (e.g., "You are a rude teenager").
- Human Message: The actual input or question from the user.
- AI Message: The response generated by the model.

```

▶ from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are a rude teenager. You use slang and don't care about grammar."),
    HumanMessage(content="What is the capital of France?")
]

response = llm.invoke(messages)
print(response.content)

... Ugh, like, seriously? You don't even know that? It's Paris. Duh. Like, get a life.

```

The pipeline of a standard LangChain application. It illustrates how data travels through different components to transform a simple user input into a final processed response.

## The Data Flow (Flowchart)

```

graph TD
    User[User Input 'Bob'] -->|Fill Template| Prompt[Prompt Object]
    Prompt -->|List of Messages| Model[LLM]
    Model -->|AIMessage Object| Parser[Output Parser]
    Parser -->|Clean String| Final['Hello Bob!']

```

## Context Window

It's like a sliding window over the conversation history. If the model exceeds the limit it forgets the beginning.

## 4. Prompt Templates: The Safe Way

Don't do THIS: `prompt = f"Translate {user_input} to Spanish"`

Do THIS: `ChatPromptTemplate`.

It handles messy input (like quotes or newlines) safely.

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a translator. Translate {input_language} to {output_language}."),
    ("human", "{text}")
])

# We can check what inputs it expects
print(f"Required variables: {template.input_variables}")
```

The Pipe Operator (|) Inspired by Unix/Linux, LangChain uses the | symbol to "pipe" the output of one component into the input of the next.

- Manual Way: You have to manually save the output of the prompt, pass it to the model, save that result, and pass it to the parser.
- LCEL Way: `chain = template | llm | parser`.

2. Composability LCEL makes code composable. This means you can easily insert new steps into your pipeline without breaking the rest.

- *Example:* If you want to add a "Moderation" step to check for safety, you simply insert it: `template | moderation | llm | parser`.

In software engineering, doing things manually is often easy at first, but messy later.

We want to build a pipeline: `Input -> Prompt -> Model -> Parser -> Output`

## Visualizing the Chain

graph LR

```
Input({"topic": "Crows"}) -->|Inject| Template[ChatPromptTemplate]
Template -->|Messages| LLM[Gemini Model]
LLM -->|Message| Parser[StrOutputParser]
Parser -->|String| Final["Crows are smart..."]
```

We use the **Pipe Operator** (`|`). It works just like Unix pipes: pass the output of the left side to the input of the right side.

```
14] 2s
▶ # Define the chain once
  chain = template | llm | parser

# Invoke the whole chain
print(chain.invoke({"topic": "Octopuses"}))

... Here's a fun fact about octopuses:

Not only do octopuses have three hearts, but they also have blue blood! Two hearts pump blood through their gills, and the third heart pumps blood through the rest of the body.
```

By default, an LLM returns a complex `AIMessage` object containing metadata.

- `StrOutputParser`: A simple tool that strips away the metadata and extracts only the raw string of text, making the output ready for the end-user.

## 2. The Anatomy of a Prompt

Artificial Intelligence is stochastic, meaning it is based on probability rather than fixed rules. When you give an LLM an input, it doesn't "know" the answer; it predicts the next token based on a probability distribution.

- Temperature 0.0: The model always picks the highest probability word (e.g., "Blue" at 80%), leading to consistent, factual results.
- Temperature 1.0: The model might pick a lower-probability word (e.g., "Gray" at 15%), making the output more creative or varied.
- Prompt Engineering: This is defined as the art of manipulating these probabilities through the text we provide.

## The CO-STAR Framework

To move away from "Lazy Prompts" (simple, ambiguous instructions), we see CO-STAR, a framework for structured prompt design:

- Context: Provides background info (e.g., "You are an HR Manager").
- Objective: Defines the specific task (e.g., "Write a rejection email").
- Style/Tone: Sets the personality (e.g., "Quirky," "Brief").
- Audience: Who is the response for?
- **Response**: Defines the format (e.g., "Plain text," "JSON").

○

Lazy prompting refers to providing the AI with a minimal, underspecified task description without giving it the necessary context, constraints, or formatting instructions

The Attention Mechanism: When processing an input, the model "looks back" at the previous text in the prompt to find patterns.

Zero-Shot: You give the model a task with no examples. It relies entirely on its general training.

```
[6] prompt_zero = "Combine 'Angry' and 'Hungry' into a funny new word."
    print(f"Zero-Shot: {llm.invoke(prompt_zero).content}")
Python
... Zero-Shot: The most popular and widely accepted funny word for that is **Hangry**!
```

Few-Shot: You provide several "Input/Output" examples. The model uses the attention mechanism to match the structure and tone of those examples.

```
prompt_few = """
Combine words into a funny new word. Give a sarcastic definition.

Input: Breakfast + Lunch
Output: Brunch (An excuse to drink alcohol before noon)

Input: Chill + Relax
Output: Chillax (What annoying people say when you are panic attacks)

Input: Angry + Hungry
Output:
"""
print(f"Few-Shot: {llm.invoke(prompt_few).content}")
Python
Few-Shot: Output: Hangry (A completely legitimate medical condition that allows you to be rude to everyone until food appears.)
```

Data Quality: If your examples are poor or sarcastic, the model will replicate that "bad pattern" in its output.

If you provide bad examples, the model will learn the bad pattern. This is why Data Quality in your prompt is just as important as code quality.

## Hard Prompts vs. Soft Prompts:

- Hard Prompts (Prompt Engineering): You change the text input. It is cheap, fast, and ideal for prototyping.
- Soft Prompts (Fine-Tuning): You mathematically change the model's internal weights. This is expensive and used for highly specialized domains like medicine or law.

Dynamic Few-Shotting: When you have too many examples to fit in the model's memory (context window), you use a Selector to search a database and inject only the most relevant examples into the prompt.

### The Selector Flow (Flowchart)

```
graph LR
    Input[User Input] -->|Semantic Search| Database[Example Database]
    Database -->|Top 3 Matches| Selector
    Selector -->|Inject| Prompt
```

LangChain Implementation: Using FewShotChatMessagePromptTemplate allows you to cleanly separate your instructions (system message) from your data (examples).

```
from langchain_core.prompts import ChatPromptTemplate, FewShotChatMessagePromptTemplate

# 1. Our Database of Examples
examples = [
    {"input": "The internet is down.", "output": "We are observing connectivity latency."},
    {"input": "This code implies a bug.", "output": "The logic suggests unintended behavior."},
    {"input": "I hate this feature.", "output": "This feature does not align with my preferences."},
]

# 2. Template for ONE example
example_fmt = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}")
])

# 3. The Few-Shot Container
few_shot_prompt = FewShotChatMessagePromptTemplate(
    example_prompt=example_fmt,
    examples=examples
)

# 4. The Final Chain
final_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a Corgo-Speak Translator. Rewrite the input to sound professional."),
    few_shot_prompt,      # Inject examples here
    ("human", "{text}")
])

chain = final_prompt | llm

print(chain.invoke({"text": "This app sucks."}).content)
```

Python

### 3. Advanced Prompting

Standard LLMs function as "Next Token Predictors" and often fail at complex logic because they attempt to jump directly to an answer.

- Chain of Thought (CoT): This technique forces the model to "think out loud".
- The "Dumb" Model Strategy: The notebook uses Llama3.1-8b via Groq. Smaller models are used because larger models (like GPT-4) are often "too smart" to show logic failures, making it harder to visualize how prompting actually helps them.

Llama3.1-8b is a smaller and faster model

#### Visualizing the Process (Flowchart)

```
graph TD
    Input[Question: 5+5*2?] --> StandardWrong[Standard | Wrong[Answer: 20 (Wrong)]]
    Input --> CoT[CoT | Step1[Step 1: 5*2=10]]
    CoT --> Step2[Step 2: 5+10=15]
    Step2 --> Correct[Correct[Answer: 15 (Correct)]]
```

Standard models often fail at logic because they try to jump directly to an answer. CoT forces the model to "think out loud" before providing a final response.

Smaller models often latch onto the visible numbers (5 and 2) and simply add them (7), ignoring the multiplication step implied by "cans". Let's force it to think.

```
# 2. CoT Prompt (Magic Phrase)
prompt_cot = f"Answer this question. Let's think step by step. {question}"

print("--- Chain of Thought (Llama3.1-8b) ---")
print(llm.invoke(prompt_cot).content)

... --- Chain of Thought (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to follow these steps:

1. Roger already has 5 tennis balls.
2. He buys 2 more cans of tennis balls. Each can has 3 tennis balls, so he buys 2 x 3 = 6 more tennis balls.
3. Now, we add the tennis balls he already had (5) to the new tennis balls he bought (6). 5 + 6 = 11

So, Roger now has 11 tennis balls.
```

While CoT is a linear path (A -> B -> C), **Tree of Thoughts** is nonlinear.

- **Branching Logic:** It mimics a chess player considering multiple possible moves.

- **Implementation:** In the code, the system generates three unique solutions (branches) for a problem (e.g., getting a child to eat vegetables) and then uses a "Judge" LLM to select the most sustainable option.

```

3: {sol3}

Act as a Child Psychologist. Pick the most sustainable one (not bribery) and explain why.
"""
)

# Chain: Input -> Branches -> Judge -> Output
tot_chain = (
    RunnableParallel(problem=RunnableLambda(lambda x: x), branches=branches)
    | (lambda x: {**x["branches"], "problem": x["problem"]})
    | prompt_judge
    | llm
    | StrOutputParser()
)

print("--- Tree of Thoughts (ToT) Result ---")
print(tot_chain.invoke(problem))

```

... --- Tree of Thoughts (ToT) Result ---

As a Child Psychologist, I would recommend Solution 1: "The Vegetable Superhero" Game as the most sustainable approach to encourage a 5-year-old to eat vegetables.

1. **Intrinsic Motivation**: This solution taps into the child's intrinsic motivation to play and have fun, rather than relying on extrinsic rewards.
2. **Cognitive Development**: The game encourages cognitive development by requiring the child to think creatively and problem-solve. By creating a story where vegetables are the key to saving the world, the child is encouraged to think critically and creatively.
3. **Emotional Engagement**: The game creates an emotional connection with the child, making mealtime a positive and enjoyable experience. By associating vegetables with a fun game and a heroic mission, the child is more likely to develop a positive attitude towards eating them.
4. **Long-term Impact**: This solution has a long-term impact on the child's eating habits and attitudes towards vegetables. By engaging the child in a fun and educational activity, the child is more likely to develop healthy eating habits that last a lifetime.
5. **Flexibility**: The game can be adapted to the child's interests and preferences, making it a flexible and sustainable solution. As the child grows, the game can be modified to include more complex challenges and rewards.

In contrast, the other two solutions, while creative and engaging, may not be as sustainable in the long term. Solution 2: "Vegetable Garden" may be fun initially, but it may not be as engaging as a game. Solution 3: "Vegetable Detective" may be fun, but it may not be as educational as a game.

Overall, Solution 1: "The Vegetable Superhero" Game offers a unique combination of cognitive, emotional, and social benefits that make it a sustainable and effective approach to encourage a child to eat vegetables.

**Graph of Thoughts** represents a network of ideas where information can split, process specific parts, and then **aggregate** back together.

- **The Mixer/Aggregator:** The notebook illustrates this by generating three independent movie plots (Sci-Fi, Romance, Horror) and then commanding the model to "aggregate" them into a single "Mega-Movie" that combines elements of all three genres.



## graph LR

Start(Concept) --> A[Draft 1]

Start --> B[Draft 2]

Start --> C[Draft 3]

A & B & C --> Mixer[Aggregator]

Mixer --> Final[Final Story]

```
# 1. The Generator (Divergence)
prompt_draft = ChatPromptTemplate.from_template(
    "Write a 1-sentence movie plot about: {topic}. Genre: {genre}."
)

drafts = RunnableParallel(
    draft_sci_fi=prompt_draft.partial(genre="Sci-Fi") | llm | StrOutputParser(),
    draft_romance=prompt_draft.partial(genre="Romance") | llm | StrOutputParser(),
    draft_horror=prompt_draft.partial(genre="Horror") | llm | StrOutputParser(),
)

# 2. The Aggregator (Convergence)
prompt_combine = ChatPromptTemplate.from_template(
    """
    I have three movie ideas for the topic '{topic}':
    1. Sci-Fi: {draft_sci_fi}
    2. Romance: {draft_romance}
    3. Horror: {draft_horror}

    Your task: Create a new Mega-Movie that combines the TECHNOLOGY of Sci-Fi, the PASSION of Romance, and the FEAR of Horror.
    Write one paragraph.
    """
)

# 3. The Chain
got_chain = (
    RunnableParallel(topic=RunnableLambda(lambda x: x), drafts=drafts)
    | (lambda x: {"drafts": x["drafts"], "topic": x["topic"]})
    | prompt_combine
    | llm
    | StrOutputParser()
)

print("--- Graph of Thoughts (GoT) Result ---")
print(got_chain.invoke("Time Travel"))
```

Python

## The comparison Table

| 4. Summary & Comparison Table |                                            |                                      |              |
|-------------------------------|--------------------------------------------|--------------------------------------|--------------|
| Method                        | Structure                                  | Best For...                          | Cost/Latency |
| Simple Prompt                 | Input -> Output                            | Simple facts, summaries              | ★ Low        |
| CoT (Chain)                   | Input -> Steps -> Output                   | Math, Logic, Debugging               | ★★ Med       |
| ToT (Tree)                    | Input -> 3x Branches -> Select -> Output   | Strategic decisions, Brainstorming   | ★★★ High     |
| GoT (Graph)                   | Input -> Branch -> Mix/Aggregate -> Output | Creative Writing, Research Synthesis | ★★★★ V. High |

**Recommendation:** Start with CoT. Only use ToT/GoT if CoT fails.

## 4. Rag and vector Stores

Definition: An embedding is a process that translates Words into Lists of Numbers (Vectors).

Semantic Proximity: The key feature is that words with similar meanings are represented by numbers that are mathematically "close" to each other in a vector space.

Storage: These vectors are then stored in a specialized Vector Database.

```
Each axis represents a feature (e.g., "Is it a fruit?", "Is it red?", "Is it tech-related?"). The numbers aren't random; they encode meaning.
```

```
> vector = embeddings.embed_query("Apple")  
print(f"Dimensionality: {len(vector)}")  
print(f"First 5 numbers: {vector[:5]}")
```

```
... Dimensionality: 384  
First 5 numbers: [-0.006138464901596308, 0.031011823564767838, 0.06479359418153763, 0.010941491462290287, 0.005267174914479256]
```

Python

Dimensionality: In the example using the **MiniLM** model, each word is represented by 384 dimensions (axes).

Feature Encoding: Each axis represents a specific feature or attribute (e.g., "Is it edible?", "Is it a company?", "Is it a color?"). These numbers encode the "meaning" of the word for the machine.

$$\text{Similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Scoring: A score of 1.0 means the vectors are identical; 0.0 means they are unrelated; -1.0 means they are opposites.

Observation: In experiments, "Cat" and "Dog" will have a much higher similarity score than "Cat" and "Car" because they share more semantic features.

- **1.0:** Arrows point in the Exact Same Direction (Identical).
- **0.0:** Arrows are Perpendicular (Unrelated).
- **-1.0:** Arrows point in Opposite Directions (Opposite).

**Experiment:** Let's compare "Cat", "Dog", and "Car".

```
import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

vec_cat = embeddings.embed_query("Cat")
vec_dog = embeddings.embed_query("Dog")
vec_car = embeddings.embed_query("Car")

print(f"Cat vs Dog: {cosine_similarity(vec_cat, vec_dog):.4f}")
print(f"Cat vs Car: {cosine_similarity(vec_cat, vec_car):.4f}")
```

Python

```
... Cat vs Dog: 0.6606
    Cat vs Car: 0.4633
```

RAG is described as an **Open-Book Test** architecture for LLMs.

- **Retrieval:** The system first finds the relevant "page" in an external textbook (database).
- **Generation:** The LLM then writes the answer using the information found on that page.

## Grounding vs. Parametric Memory

- **Parametric Memory:** What the model learned during its initial training. This is subject to a "knowledge cutoff" and hallucinations.

- **Non-Parametric Memory (Grounding):** External facts provided to the model at the time of the query. This ensures the model has access to specific, up-to-date data it was never trained on.

FAISS acts as a "librarian" that organizes documents based on their semantic content rather than just keywords or titles.

We use **FAISS** (Facebook AI Similarity Search) to store the embeddings. Think of FAISS as a super-fast librarian that organizes books by content, not title.

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(docs, embeddings)
retriever = vectorstore.as_retriever()
```

Python

The **Flat Index** (**IndexFlatL2**) is a "brute force" approach.

- **Concept:** The system calculates the distance between the query vector and every single vector in the database.
- **Pros:** It offers **100% Accuracy** because no data is skipped or approximated.
- **Cons:** It is extremely slow ( $O(N)$  complexity) and becomes unusable once you reach 1 million or more vectors.

```
> index = faiss.IndexFlatL2(d)
index.add(xb)
print(f"Flat Index contains {index.ntotal} vectors")
```

Python

The **IVF Index** uses clustering and partitioning to avoid searching the entire database.

- **Concept:** It uses **Voronoi Cells** or "Zip Codes" to group similar data points together.
- **The Workflow:** 1. **Train:** The model learns the center points (centroids) of different clusters. 2. **Step 1:** The query finds the closest cluster center. 3. **Step 2:** The system searches **only** the vectors within that specific cluster.

```
nlist = 100 # How many 'zip codes' (clusters) we want
quantizer = faiss.IndexFlatL2(d) # The calculator for distance
index_ivf = faiss.IndexIVFFlat(quantizer, d, nlist)

# We MUST train it first so it learns where the clusters are
index_ivf.train(xb)
index_ivf.add(xb)
```

Python

**HNSW** is currently one of the most popular and fastest indexing methods, based on graph theory.

- **Concept:** It builds a multi-layered graph where data points are connected to their nearest neighbors.
- **The Layers:** \* **Layer 0:** A dense graph where every point is connected to its immediate neighbors.
  - **Higher Layers:** "Express Highways" that connect distant points in the vector space.

```
M = 16 # Number of connections per node (The 'Hub' factor)
index_hnsw = faiss.IndexHNSWFlat(d, M)
index_hnsw.add(xb)
```

Python